

Day to Night Scene



practical 1 : Turn day scene into a night scene on mouse click in Unity
give detailed steps
my version is 2022.3

Here are **step-by-step detailed instructions** for **Practical 1: Turn Day Scene into Night Scene on Mouse Click in Unity (version 2022.3)**.

I'll assume you are a complete beginner and explain **every click**.

Practical 1: Turn Day Scene into Night Scene on Mouse Click

What we will make

Initially:

☀ Scene = Day

When mouse is clicked:

🌙 Scene = Night

When clicked again:

☀ Back to Day

(toggle effect)

Step 1 — Open Unity Hub

Open Unity Hub.

Click **New Project**

Choose:

- **3D Core**

Project name:

DayNightScene

Choose location.

Click **Create Project**

Wait for editor to open.

Step 2 — Understand screen

You will see:

- **Hierarchy** → left side
- **Scene View** → center
- **Inspector** → right side
- **Project Folder** → bottom

We mainly use these 4 panels.

Step 3 — Create Ground

In top menu:

GameObject → **3D Object** → **Plane**

Now plane appears.

Rename:

Ground

How:

Hierarchy → right click Plane → Rename → Ground

Reset transform:

Inspector:

Position:

X = 0
Y = 0
Z = 0

Rotation:

0 0 0

Scale:

1 1 1

Step 4 — Add Some Objects

Make scene look nice.

Create cube:

GameObject → **3D Object** → **Cube**

Rename:

House

Position:

X = 0
Y = 0.5
Z = 0

Scale:

X = 2
Y = 1
Z = 2

Create sphere:

GameObject → **3D Object** → **Sphere**

Rename:

TreeTop

Position:

X = 3
Y = 1.5
Z = 0

Create cylinder:

GameObject → **3D Object** → **Cylinder**

Rename:

TreeStem

Position:

X = 3
Y = 1
Z = 0

Scale:

X = 0.3
Y = 1
Z = 0.3

Now scene looks better.

Step 5 — Check Directional Light

Hierarchy contains:

Directional Light

This acts like Sun.

Select it.

Inspector:

Light Type:

Directional

Intensity:

1

Color:

white

Keep as it is.

Step 6 — Create Sky Color Script

Bottom panel:

Project → Assets

Right click

Create → C# Script

Name:

DayNightToggle

Double click script.

Delete everything.

Paste this:

```
C#
using UnityEngine;

public class DayNightToggle : MonoBehaviour
{
    public Light sun;

    bool isNight = false;

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            if (isNight == false)
            {
                // NIGHT
                RenderSettings.ambientLight = new Color(0.1f, 0.1f, 0.3f);
                Camera.main.backgroundColor = new Color(0.02f, 0.02f, 0.1f);

                sun.intensity = 0.2f;

                isNight = true;
            }
            else
            {
                // DAY
                RenderSettings.ambientLight = Color.white;
                Camera.main.backgroundColor = new Color(0.5f, 0.8f, 1f);

                sun.intensity = 1f;

                isNight = false;
            }
        }
    }
}
```

Save file:

Ctrl + S

Return to Unity.

Wait for compile.

Step 7 — Create Empty Object

Hierarchy:

Right click

Create Empty

Rename:

GameManager

Step 8 — Attach Script

Drag script:

DayNightToggle

onto

GameManager

OR

Select GameManager → Add Component → search:

DayNightToggle

Add it.

Step 9 — Assign Light

Script has:

Sun
(None)

Drag:

Hierarchy → Directional Light

into Sun box.

Done.

Step 10 — Set Camera Background

Click:

Main Camera

Inspector:

Camera component →

Background color →

choose sky blue.

Example:

light blue.

This is daytime sky.

Step 11 — Play

Press top center:

 Play

Click mouse in Game window.

Result:

Click 1:

☀ Day → 🌙 Night

Click 2:

🌙 Night → ☀ Day

Click 3:

☀ Day → 🌙 Night

continues toggling.

How code works

When clicked:

C#

`Input.GetMouseButton(0)`

means:

Left mouse clicked.

Night mode:

Changes:

- ambient light darker
 - camera sky dark blue
 - sun intensity low
-

Day mode:

Changes:

- bright light
- blue sky

- strong sunlight

If not working

Check:

1 Script attached?

Must be on GameManager.

2 Sun assigned?

Directional Light must be dragged into script box.

3 Click inside Game window

Not Scene window.

4 No compile errors

Console should be empty.

Viva Questions

Q1 What is Directional Light?

Sun-like light source.

Q2 What is Update()?

Runs every frame.

Q3 What is Input.GetMouseButtonDown(0)?

Detects left mouse click.

Q4 What is RenderSettings?

Controls scene lighting.

Q5 What is Camera.main?

Main camera in scene.

This practical can also be done **without script** using **Animation + Timeline**, but for viva/practical exam, this scripting method is the easiest and most reliable.



2.Create a scene, where a candle is lit (with shadows and smoke), on mouse click using Unity

Here's **Practical 2** in detailed beginner-friendly steps for **Unity**.

Practical 2: Create a scene where a candle is lit (with shadows and smoke) on mouse click

What we will make

Initially:

 Candle is OFF

On mouse click:

 Flame appears

 Smoke appears

 Light glows

 Shadow appears

Click again:

Candle OFF

(toggle ON/OFF)

Step 1 — Create New Project

Open Unity Hub → **New Project**

Choose:

- **3D Core**

Project Name:

CandleScene

Click **Create Project**

Step 2 — Create Table / Ground

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Table

Keep position:

X=0 Y=0 Z=0

Step 3 — Create Candle Body

Top menu:

GameObject → **3D Object** → **Cylinder**

Rename:

Candle

Set:

Position:

- X = 0
- Y = 1
- Z = 0

Scale:

- X = 0.3
- Y = 1
- Z = 0.3

This becomes candle wax.

Step 4 — Make Candle White

Select Candle.

Inspector → **Mesh Renderer** → **Materials**

Color = White / Cream

Done.

Step 5 — Create Wick

GameObject → **3D Object** → **Cylinder**

Rename:

Wick

Set:

Position:

- X = 0
- Y = 2.1

- $Z = 0$

Scale:

- $X = 0.03$
- $Y = 0.1$
- $Z = 0.03$

Make color black.

Done.

Step 6 — Create Flame

GameObject → **3D Object** → **Sphere**

Rename:

Flame

Set:

Position:

- $X = 0$
- $Y = 2.35$
- $Z = 0$

Scale:

- $X = 0.15$
- $Y = 0.25$
- $Z = 0.15$

Material color:

Orange / Yellow

Done.

Step 7 — Add Light

GameObject → **Light** → **Point Light**

Rename:

CandleLight

Set:

Position:

- $X = 0$
- $Y = 2.4$
- $Z = 0$

Inspector:

Range:

6

Intensity:

3

Color:

Warm yellow

Enable shadows:

Light → Shadows → **Soft Shadows**

Done.

Now candle casts shadow.

Step 8 — Create Smoke

Hierarchy → Right click

Effects → **Particle System**

Rename:

Smoke

Move:

$X=0$ $Y=2.6$ $Z=0$

Inspector settings:

Duration:

5

Looping = ON

Start Lifetime:

2

Start Speed:

0.5

Start Size:

0.2

Start Color:

Gray

Shape:

Cone → very small radius

Simulation Space:

World

This makes smoke rise upward.

Step 9 — Turn OFF Flame / Light / Smoke Initially

Disable:

- Flame
- CandleLight
- Smoke

How:

Uncheck checkbox in Inspector (top-left of component object)

Leave Candle and Wick ON.

Initial scene:

Unlit candle.

Step 10 — Create Script

Project window:

Right click → **Create** → **C# Script**

Name:

CandleToggle

Open it.

Delete old code.

Paste:

```
C#  
using UnityEngine;
```

```
public class CandleToggle : MonoBehaviour
{
    public GameObject flame;
    public GameObject smoke;
    public Light candleLight;

    bool isLit = false;

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            isLit = !isLit;

            flame.SetActive(isLit);
            smoke.SetActive(isLit);
            candleLight.enabled = isLit;
        }
    }
}
```

Save → Ctrl+S

Return to Unity.

Wait compile.

Step 11 — Create Manager

Hierarchy → Right click → **Create Empty**

Rename:

GameManager

Add component:

CandleToggle

Step 12 — Assign Objects

Drag:

Flame → flame slot

Smoke → smoke slot

CandleLight → candleLight slot

Done.

Step 13 — Play

Press  Play

Click mouse.

Result:

Click 1:

 →  Lit candle
 glow
 shadow
 smoke

Click 2:

Everything OFF

Click 3:

ON again

Viva Questions

Q1 What is Particle System?

Used for smoke/fire/rain effects.

Q2 Why Point Light?

Light spreads in all directions like flame.

Q3 Why Shadows?

Makes scene realistic.

Q4 What is SetActive()?

Shows/hides object.

Q5 What is enabled property?

Turns component ON/OFF.

This practical looks impressive in exam because it includes **lighting + shadows + smoke + interaction**.



Change a character emotion (Neutral to happy, Happy to sad, sad to angry etc.,) using Unity.

Here is **Practical 3** for **Change character emotion (Neutral → Happy → Sad → Angry) on mouse click in Unity**, explained step by step for a beginner.

We'll do it the **easy practical way** (simple face changes), no advanced 3D character modeling needed.

What we will make

Initial:

 Neutral

Click mouse:

😊 Happy

Click again:

😞 Sad

Click again:

😡 Angry

Click again:

😊 Neutral

Cycle repeats.

Step 1 — Open Project

Open Unity Hub

Create:

- **3D Core Project**

Name:

CharacterEmotion

Click **Create**

Step 2 — Create Ground

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Ground

Step 3 — Create Character Head

Top menu:

GameObject → **3D Object** → **Sphere**

Rename:

Face

Set Position:

- $X = 0$

- $Y = 2$
- $Z = 0$

Scale:

- $X = 2$
- $Y = 2$
- $Z = 2$

This becomes face.

Step 4 — Create Eyes

Create sphere:

GameObject → **3D Object** → **Sphere**

Rename:

LeftEye

Scale:

- 0.15
- 0.15
- 0.15

Position:

- $X = -0.35$
- $Y = 2.25$
- $Z = 0.9$

Make black.

Duplicate:

Ctrl + D

Rename duplicate:

RightEye

Move:

- $X = 0.35$
- $Y = 2.25$
- $Z = 0.9$

Done.

Step 5 — Create 4 Mouth Objects

We make 4 mouths and show one at a time.

Neutral Mouth

GameObject → 3D Object → Cube

Rename:

NeutralMouth

Scale:

- X = 0.5
- Y = 0.08
- Z = 0.05

Position:

- X = 0
- Y = 1.7
- Z = 0.95

Black color.

Happy Mouth

Duplicate NeutralMouth.

Rename:

HappyMouth

Move slightly up:

Y = 1.8

Scale X = 0.6

(Optional: slightly rotate for smile)

Sad Mouth

Duplicate again.

Rename:

SadMouth

Move:

Y = 1.6

Rotate X slightly.

Angry Mouth

Duplicate again.

Rename:

AngryMouth

Scale X:

0.7

Done.

Step 6 — Angry Eyebrows

Create Cube.

Rename:

LeftBrow

Scale:

- X = 0.3
- Y = 0.05
- Z = 0.05

Position:

- X = -0.35
- Y = 2.55
- Z = 0.95

Rotate Z:

20

Duplicate:

Rename:

RightBrow

Move X:

0.35

Rotate Z:

-20

Initially disable eyebrows.

Step 7 — Disable Extra Objects Initially

Keep ON:

☒ NeutralMouth

Disable:

☒ HappyMouth

☒ SadMouth

☒ AngryMouth

☒ LeftBrow

☒ RightBrow

Step 8 — Create Script

Project → Right click → Create → C# Script

Name:

EmotionChange

Open script.

Replace code:

```
C#
using UnityEngine;

public class EmotionChange : MonoBehaviour
{
    public GameObject neutralMouth;
    public GameObject happyMouth;
    public GameObject sadMouth;
    public GameObject angryMouth;
    public GameObject leftBrow;
    public GameObject rightBrow;

    int emotion = 0;

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            emotion++;

            if (emotion > 3)
```

```

        emotion = 0;

        ShowEmotion();
    }
}

void ShowEmotion()
{
    neutralMouth.SetActive(false);
    happyMouth.SetActive(false);
    sadMouth.SetActive(false);
    angryMouth.SetActive(false);
    leftBrow.SetActive(false);
    rightBrow.SetActive(false);

    if (emotion == 0)
    {
        neutralMouth.SetActive(true);
    }
    else if (emotion == 1)
    {
        happyMouth.SetActive(true);
    }
    else if (emotion == 2)
    {
        sadMouth.SetActive(true);
    }
    else if (emotion == 3)
    {
        angryMouth.SetActive(true);
        leftBrow.SetActive(true);
        rightBrow.SetActive(true);
    }
}
}

```

Save.

Return to Unity.

Step 9 — Create Empty Object

Hierarchy → Right click → Create Empty

Rename:

GameManager

Add Component:

EmotionChange

Step 10 — Assign References

Drag all objects into script slots:

- NeutralMouth
- HappyMouth
- SadMouth

- AngryMouth
- LeftBrow
- RightBrow

Done.

Step 11 — Play

Press 

Mouse click changes:

- 😊 Neutral
- 😊 Happy
- 😞 Sad
- 😡 Angry
- 😊 Neutral

repeat.

Viva Questions

Q1 Why SetActive()?

Show/hide objects.

Q2 Why use GameObject references?

To control scene objects.

Q3 What is Update()?

Runs every frame.

Q4 What is Input.GetMouseButtonDown(0)?

Detect mouse click.

Q5 What is emotion variable?

Stores current emotion state.

This is the **simplest exam-friendly method** for emotion changing in Unity 2022.3.



Depict a scene of the effect of kicking a ball in Unity (at 3 various speeds).

Here is **Practical 4: Depict effect of kicking a ball at 3 different speeds in Unity**, explained in detailed beginner steps.

Goal:

Mouse click changes kick power:

1st click → Slow kick 

2nd click → Medium kick  

3rd click → Fast kick   

Then repeat.

We will use **Physics (Rigidbody + Force)**.

Step 1 — Create New Project

Open Unity Hub

Create:

- **3D Core**

Project name:

KickBall

Click **Create**

Step 2 — Create Ground

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Ground

Keep default transform.

Step 3 — Create Ball

Top menu:

GameObject → **3D Object** → **Sphere**

Rename:

Ball

Set:

Position:

- X = 0
- Y = 0.5

- $Z = 0$

Scale:

- 1
- 1
- 1

Step 4 — Add Rigidbody

Select Ball.

Inspector →

Add Component

Search:

Rigidbody

Add it.

This gives physics.

Gravity stays ON.

Step 5 — Add Material

Make ball visible.

Inspector → Material → choose any color.

Example:

White.

Step 6 — Create Goal Marker (optional)

GameObject → 3D Object → Cube

Rename:

Goal

Position:

- $X = 0$
- $Y = 1$
- $Z = 20$

Scale:

- X = 5
- Y = 2
- Z = 0.5

This shows where ball goes.

Step 7 — Create Script

Project → Right click →

Create → C# Script

Name:

KickBall

Open script.

Delete old code.

Paste:

```
C#
using UnityEngine;

public class KickBall : MonoBehaviour
{
    Rigidbody rb;

    int speedLevel = 0;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            rb.linearVelocity = Vector3.zero;
            rb.angularVelocity = Vector3.zero;

            transform.position = new Vector3(0, 0.5f, 0);

            speedLevel++;

            if (speedLevel > 3)
                speedLevel = 1;

            float force = 0;

            if (speedLevel == 1)
                force = 200f;    // slow

            if (speedLevel == 2)
                force = 400f;    // medium

            if (speedLevel == 3)
                force = 700f;    // fast

            rb.AddForce(Vector3.forward * force);
        }
    }
}
```


Save.

Return to Unity.

Step 8 — Attach Script

Drag script onto Ball.

OR:

Ball → Add Component → search:

KickBall

Add.

Done.

Step 9 — Camera Position

Select Main Camera.

Set:

Position:

- X = 0
- Y = 5
- Z = -10

Rotation:

- X = 20
- Y = 0
- Z = 0

Now camera sees ball nicely.

Step 10 — Play

Press 

Click mouse:

First click

Slow:

 rolls slowly

Second click

Medium:

⚽ faster

Third click

Fast:

⚽⚽ very fast

Fourth click repeats slow.

How it works

Script changes:

200 force

↓

400 force

↓

700 force

Then repeats.

Physics moves ball.

Viva Questions

Q1 What is Rigidbody?

Adds physics.

Q2 What is AddForce()?

Applies push force.

Q3 What is Vector3.forward?

Forward direction.

Q4 Why reset position?

Bring ball back before next kick.

Q5 Why angularVelocity = 0?

Stops previous spin.

One small note for **Unity 2022.3**: if `linearVelocity` gives error, replace:

```
C#
rb.linearVelocity = Vector3.zero;
```

with:

```
C#
rb.velocity = Vector3.zero;
```

because most versions use **velocity**.



Simulate bouncing ball in Unity. The bouncing should stop gradually after the time specified by the user

Here is **Practical 5: Simulate a bouncing ball in Unity, where bouncing gradually stops after time specified by user**, in beginner-friendly detailed steps.

Goal:

- Ball starts bouncing 🏀
- User sets time (example: 10 sec)
- Bounce becomes smaller gradually
- Finally stops bouncing

This looks realistic because energy reduces slowly.

We will use:

- ✓ Rigidbody Physics
- ✓ Bounce Material
- ✓ Timer
- ✓ Gradual damping

Step 1 — Create New Project

Open Unity Hub

Create:

- **3D Core**

Project name:

BouncingBall

Click **Create**

Step 2 — Create Ground

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Ground

Keep default position.

Step 3 — Create Ball

Top menu:

GameObject → **3D Object** → **Sphere**

Rename:

Ball

Position:

- X = 0
- Y = 5
- Z = 0

This starts high above ground.

Step 4 — Add Rigidbody

Select Ball.

Inspector → **Add Component**

Search:

Rigidbody

Add.

Gravity = ON

Done.

Step 5 — Create Bouncy Material

Project window →

Right click →

Create → **Physic Material**

Name:

BounceMaterial

Select it.

Inspector settings:

Dynamic Friction:

0

Static Friction:

0

Bounciness:

1

Bounce Combine:

Maximum

Done.

Step 6 — Apply Material

Apply to Ball:

Select Ball →

Sphere Collider →

Material →

drag:

BounceMaterial

into slot.

Done.

Now ball bounces.

Step 7 — Create Script

Project →

Right click →

Create → C# Script

Name:

BounceControl

Open script.

Delete old code.

Paste:

```
C#
using UnityEngine;

public class BounceControl : MonoBehaviour
{
    public float stopAfterSeconds = 10f;

    Rigidbody rb;
    float timer;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        timer = stopAfterSeconds;
    }

    void Update()
    {
        timer -= Time.deltaTime;

        // gradually reduce bounce
        rb.velocity = new Vector3(
            rb.velocity.x * 0.995f,
            rb.velocity.y,
            rb.velocity.z * 0.995f
        );

        // stop after specified time
        if (timer <= 0)
        {
            rb.velocity = Vector3.zero;
            rb.angularVelocity = Vector3.zero;
            rb.useGravity = false;
            enabled = false;
        }
    }
}
```

Save:

Ctrl + S

Return to Unity.

Step 8 — Attach Script

Drag script onto Ball

OR

Ball → Add Component →

Search:

BounceControl

Add.

Done.

Step 9 — Set User Time

In Inspector you will see:

Stop After Seconds = 10

User can change:

Examples:

5 → stops in 5 sec

10 → stops in 10 sec

20 → stops in 20 sec

This satisfies:

"time specified by user"

Step 10 — Improve Bounce (Initial Push)

Select Ball →

Rigidbody →

set:

Mass:

1

Drag:

0

Angular Drag:

0.05

Collision Detection:

Continuous

Looks smoother.

Step 11 — Play

Press 

Result:

Initially:

BIG bounce  

Then:

medium bounce  

Then:

small bounce  

Finally:

stops

Realistic motion.

How code works

Timer decreases:

```
C#
timer -= Time.deltaTime;
```

When reaches zero:

ball stops.

Velocity slowly reduces:

```
C#
0.995f
```

This removes energy gradually.

So bounce becomes smaller.

Viva Questions

Q1 What is Rigidbody?

Adds physics.

Q2 What is Physic Material?

Controls friction and bounce.

Q3 What is Bounciness?

How much object rebounds.

Q4 What is Time.deltaTime?

Time per frame.

Q5 Why velocity = zero?

Stops movement.

Small correction for Unity 2022.3

If needed, use:

```
C#
rb.velocity
```

(not `linearVelocity`)

because `velocity` is standard in your version.

This practical is very commonly asked because it demonstrates **physics + timer + damping** together.



Create 3D model of Reading Table and props required on it and set up the Lighting for the same using Unity

Here is **Practical 6: Create 3D model of Reading Table and props on it + set up lighting in Unity**, explained in detailed beginner-friendly steps.

Goal scene:

 Reading Table Scene

Objects:

- ✓ Table
- ✓ Chair
- ✓ Books
- ✓ Lamp
- ✓ Pen holder
- ✓ Notebook/Laptop
- ✓ Proper lighting + shadows

This looks like a complete study desk scene.

Step 1 — Create New Project

Open Unity Hub

Create:

- **3D Core**

Project name:

ReadingTableScene

Click **Create**

Step 2 — Create Floor

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Floor

Keep default transform.

Material:

light brown / wooden.

Step 3 — Create Table Top

Top menu:

GameObject → **3D Object** → **Cube**

Rename:

TableTop

Set:

Position:

- X = 0
- Y = 2
- Z = 0

Scale:

- X = 4
- Y = 0.2
- Z = 2

Now table surface is ready.

Step 4 — Create Table Legs

Create cube.

Rename:

Leg1

Scale:

- X = 0.2
- Y = 2
- Z = 0.2

Position:

- X = -1.8
- Y = 1
- Z = -0.8

Duplicate 3 times:

Ctrl + D

Rename:

- Leg2
- Leg3
- Leg4

Set positions:

Leg2:

- $X = 1.8$
- $Y = 1$
- $Z = -0.8$

Leg3:

- $X = -1.8$
- $Y = 1$
- $Z = 0.8$

Leg4:

- $X = 1.8$
- $Y = 1$
- $Z = 0.8$

Table complete.

Step 5 — Create Chair

Seat:

Cube

Rename:

ChairSeat

Position:

- $X = 0$
- $Y = 1$
- $Z = -2$

Scale:

- $X = 1.5$
- $Y = 0.2$
- $Z = 1.5$

Backrest:

Cube

Rename:

ChairBack

Position:

- $X = 0$
- $Y = 1.8$
- $Z = -2.7$

Scale:

- $X = 1.5$
- $Y = 1.5$
- $Z = 0.2$

Chair done.

Step 6 — Create Books

Cube →

Rename:

Book1

Position:

- $X = -1$
- $Y = 2.2$
- $Z = 0$

Scale:

- $X = 0.7$
- $Y = 0.1$
- $Z = 1$

Duplicate 2–3 books.

Stack them.

Change colors:

- red
- blue
- green

Looks realistic.

Step 7 — Create Notebook / Laptop

Cube →

Rename:

Notebook

Position:

- $X = 0$
- $Y = 2.2$
- $Z = 0.4$

Scale:

- $X = 1.5$
- $Y = 0.05$
- $Z = 1$

Gray/black material.

Step 8 — Create Pen Holder

Cylinder →

Rename:

PenHolder

Position:

- $X = 1.2$
- $Y = 2.3$
- $Z = 0$

Scale:

- $X = 0.2$

- $Y = 0.3$
- $Z = 0.2$

Pens:

thin cylinders.

Put 2-3 inside.

Step 9 — Create Table Lamp

Base:

Cylinder

Rename:

LampBase

Position:

- $X = 1.5$
- $Y = 2.2$
- $Z = -0.5$

Scale:

0.2 / 0.1 / 0.2

Pole:

Cylinder

$Y = 2.8$

Thin.

Top lamp:

Sphere / capsule.

Looks like desk lamp.

Step 10 — Add Lamp Light

Top menu:

GameObject → **Light** → **Spot Light**

Rename:

DeskLight

Position:

- $X = 1.5$
- $Y = 3.4$
- $Z = -0.5$

Rotate:

- $X = 60$

Inspector:

Range:

10

Intensity:

6

Spot Angle:

45

Color:

warm yellow.

Shadows:

Soft Shadows

This becomes reading lamp light.

Step 11 — Scene Light

Select:

Directional Light

Set intensity:

0.4

Soft ambient room lighting.

Lamp becomes focus.

Step 12 — Camera

Main Camera:

Position:

- $X = 0$
- $Y = 4$
- $Z = -8$

Rotation:

- $X = 20$

Good viewing angle.

Step 13 — Beautify

Optional:

Add wall:

Cube behind table.

Add carpet:

Flat cube.

Wood material on table.

Metal material on lamp.

Books colorful.

Scene becomes attractive.

Step 14 — Play

Press 

You now have:

- ☒ 3D reading table
- ☒ chair
- ☒ books
- ☒ notebook
- ☒ pen holder
- ☒ lamp
- ☒ realistic lighting
- ☒ shadows

Practical complete.

Viva Questions

Q1 Why Spot Light?

Focus light in one direction.

Q2 Why Directional Light?

Ambient room light.

Q3 Why Soft Shadows?

Realistic shadow edges.

Q4 Why Materials?

Give realistic surface appearance.

Q5 What are props?

Objects placed in scene (books, lamp, pen holder, etc.)

This is one of the easiest modeling practicals because everything can be built using **Cube + Sphere + Cylinder + Light** only.



Design a scene that allows the user to place a character on a table and interact with it in Unity.

Here is **Practical 7: Design a scene that allows user to place a character on a table and interact with it in Unity**, in detailed beginner-friendly steps.

Goal:

Create scene with:



Table



Character placed on table



Mouse click interaction:

- move
- rotate
- jump / wave (simple interaction)

This looks interactive in viva.

We will use simple 3D objects (capsule character), because importing characters is extra work.

Step 1 — Create New Project

Open Unity Hub

Create:

- **3D Core**

Project name:

CharacterOnTable

Click **Create**

Step 2 — Create Floor

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Floor

Done.

Step 3 — Create Table

Create cube:

GameObject → **3D Object** → **Cube**

Rename:

TableTop

Set:

Position:

- X = 0
- Y = 2
- Z = 0

Scale:

- X = 4
 - Y = 0.2
 - Z = 2
-

Create 4 legs (duplicate cube).

Leg positions:

(-1.8,1,-0.8)

(1.8,1,-0.8)

(-1.8,1,0.8)

(1.8,1,0.8)

Scale:

0.2 / 2 / 0.2

Table complete.

Step 4 — Create Character

Top menu:

GameObject → **3D Object** → **Capsule**

Rename:

Character

Position:

- X = 0
- Y = 3
- Z = 0

(Character stands on table)

Scale:

- 0.6
- 0.8
- 0.6

Make color blue/red.

Step 5 — Add Eyes (optional)

Two tiny spheres.

Makes character look nice.

Step 6 — Create Script

Project →

Right click →

Create → C# Script

Name:

CharacterInteract

Open script.

Replace code:

```
C#  
using UnityEngine;  
  
public class CharacterInteract : MonoBehaviour  
{
```

```
int state = 0;
Vector3 startPos;

void Start()
{
    startPos = transform.position;
}

void Update()
{
    if (Input.GetMouseButtonDown(0))
    {
        state++;

        if (state > 3)
            state = 1;

        if (state == 1)
        {
            // move
            transform.position += new Vector3(1f, 0, 0);
        }
        else if (state == 2)
        {
            // rotate
            transform.Rotate(0, 90, 0);
        }
        else if (state == 3)
        {
            // jump
            transform.position = startPos + new Vector3(0, 1f, 0);
        }
    }

    if (Input.GetMouseButtonDown(1))
    {
        transform.position = startPos;
        transform.rotation = Quaternion.identity;
    }
}
}
```

Save.

Return Unity.

Step 7 — Attach Script

Drag script onto:

Character

Done.

Step 8 — Lighting

GameObject →

Light →

Directional Light

Keep default.

Intensity:

1.2

Shadows:

Soft Shadows.

Step 9 — Camera

Main Camera:

Position:

- $X = 0$
- $Y = 5$
- $Z = -8$

Rotation:

- $X = 25$

Nice view.

Step 10 — Play

Press 

Left click:

1st click:

 move

2nd click:

 rotate

3rd click:

 jump

4th click:

repeat

Right click:

reset

Optional Better Interaction

You can also make:

- character wave
- change color

- dance spin
- walk

But above is easiest for practical.

Viva Questions

Q1 Why Capsule?

Simple human body model.

Q2 What is Transform?

Controls position/rotation/scale.

Q3 What is Quaternion.identity?

Default rotation.

Q4 What is Vector3?

Stores X Y Z coordinates.

Q5 Why save startPos?

To reset character.

This practical is good because it combines:

- ✓ modeling
- ✓ placement
- ✓ user interaction
- ✓ scripting



Create a basic 2D character controller. Move the player character using Unity's Input System.

Here is **Practical 8: Create a basic 2D Character Controller using Unity's Input System in Unity**, explained step by step for beginners.

Goal:

Control a 2D player:

- ← Left move
- Right move
- ↑ Jump

(Arrow keys / WASD)

Uses **Unity Input System** (new system).

Step 1 — Create Project

Open Unity Hub

Click **New Project**

Choose:

- **2D Core**

Project name:

CharacterController2D

Click **Create**

Step 2 — Install Input System

Top menu:

Window → **Package Manager**

Packages:

Unity Registry

Search:

Input System

Select package.

Click:

Install

Wait.

Unity asks:

Restart Editor

Click:

Yes

Project reopens.

Step 3 — Enable Input System

Go:

Edit → **Project Settings** → **Player**

Find:

Active Input Handling

Change:

Both

or

Input System Package (New)

Restart Unity if asked.

Done.

Step 4 — Create Ground

Hierarchy →

Right click →

2D Object → Sprite → Square

Rename:

Ground

Scale:

- X = 10
- Y = 1
- Z = 1

Position:

- X = 0
- Y = -3
- Z = 0

Color:

Green/Brown.

Step 5 — Create Player

Hierarchy →

Right click →

2D Object → Sprite → Square

Rename:

Player

Scale:

1 / 1 / 1

Position:

0 / 0 / 0

Color:

Blue/Red.

Step 6 — Add Physics

Select Player.

Add Component:

Rigidbody 2D

Gravity Scale:

3

Add:

Box Collider 2D

Done.

Ground:

Add:

Box Collider 2D

Done.

Step 7 — Create Input Actions

Project window →

Right click →

Create →

Input Actions

Name:

PlayerControls

Double click it.

Editor opens.

Create Action Map:

Name:

Player

Add Action:

Move

Type:

Value

Control Type:

Vector2

Binding:

Add **2D Vector Composite**

Map:

- Up → W
- Down → S
- Left → A
- Right → D

Add another binding:

Arrow keys.

Done.

Create Action:

Jump

Type:

Button

Bind:

Space

Save asset.

Step 8 — Generate C# Class

Inspector of PlayerControls →

Check:

Generate C# Class

Click:

Apply

Unity creates class.

Done.

Step 9 — Create Script

Create C# Script:

PlayerMovement

Paste:

```
C#
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 5f;
    public float jumpForce = 8f;

    Rigidbody2D rb;
    PlayerControls controls;

    Vector2 moveInput;

    void Awake()
    {
        controls = new PlayerControls();

        controls.Player.Move.performed += ctx =>
            moveInput = ctx.ReadValue<Vector2>();

        controls.Player.Move.canceled += ctx =>
            moveInput = Vector2.zero;

        controls.Player.Jump.performed += ctx =>
            Jump();
    }

    void OnEnable()
    {
        controls.Enable();
    }

    void OnDisable()
    {
        controls.Disable();
    }

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    void Update()
    {
        rb.linearVelocity =
            new Vector2(moveInput.x * moveSpeed, rb.linearVelocity.y);
    }

    void Jump()
    {
        rb.linearVelocity =
            new Vector2(rb.linearVelocity.x, jumpForce);
    }
}
```

Save.

Step 10 — Attach Script

Drag script onto Player.

Done.

Step 11 — Camera

Main Camera:

Position:

- X = 0
- Y = 0
- Z = -10

Done.

Step 12 — Play

Press 

Controls:

WASD / Arrow keys:

  Move

Space:

 Jump

Character moves.

Practical complete.

If error comes

In **Unity 2022.3**, if:

`linearVelocity`

errors,

replace all:

C#
`rb.linearVelocity`

with:

C#
`rb.velocity`

That works.

Viva Questions

Q1 What is Input System?

New input package of Unity.

Q2 Why Input Actions?

Maps controls easily.

Q3 Why Rigidbody2D?

2D physics.

Q4 What is Vector2?

X,Y coordinate pair.

Q5 What is callback context?

Gets input value.

This practical is important because many Unity labs ask **Input System specifically**, not old Input Manager.



VR Meditation Environment - Create a peaceful VR scene (waterfall, nature sounds, skybox).

Here is **Practical 9: VR Meditation Environment (Waterfall + Nature Sounds + Skybox) in Unity**, explained step by step for beginners.

Goal scene:



Trees



Waterfall



Beautiful skybox



Nature sounds (birds / water)



Soft lighting



VR-ready environment

This practical looks impressive in viva.

We'll make it **simple and practical**, without heavy modeling.

Step 1 — Create Project

Open Unity Hub

Create:

- **3D Core**

Project name:

VRMeditation

Click **Create**

Step 2 — Install XR Plugin

Top menu:

Window → **Package Manager**

Search:

XR Plugin Management

Install.

Go:

Edit → **Project Settings** → **XR Plugin Management**

Install plugin.

Enable:

- OpenXR

Done.

(Enough for basic VR setup.)

Step 3 — Create Terrain

Hierarchy →

Right click →

3D Object → **Terrain**

Rename:

Ground

Set terrain size larger.

Example:

500 x 500

Step 4 — Make Hills

Select Terrain.

Inspector →

Paint Terrain →

Raise/Lower Terrain

Create:

- small hills
- uneven land
- meditation valley

Looks natural.

Step 5 — Add Grass / Texture

Paint Texture:

Add layers:

- grass
- soil
- rock

Terrain becomes realistic.

Step 6 — Create Waterfall Rock

Create cubes / rocks near hill.

Arrange cliff shape.

Rename:

WaterfallRock

Step 7 — Create Waterfall

Easy method:

GameObject →

3D Object → Plane

Rotate vertically.

Scale tall.

Apply blue-white transparent material.

Move downward.

Looks like falling water.

Alternative:

Use Particle System for better waterfall.

Step 8 — Create River / Pond

Create Plane.

Rename:

Water

Blue transparent material.

Place under waterfall.

Done.

Step 9 — Add Trees

Hierarchy →

Terrain →

Paint Trees

Add:

- trees
- bushes
- plants

Place around.

Scene becomes peaceful.

Step 10 — Add Skybox

Go:

Window → Rendering → Lighting

Environment →

Skybox Material

Choose:

- blue sky
or
- sunset sky

Beautiful background added.

Step 11 — Add Audio

Project →

Import audio:

- birds chirping
- waterfall sound
- wind sound

(Download free sounds from official asset sources or your own files.)

Create Empty object:

Rename:

NatureSound

Add Component:

Audio Source

Assign clip.

Check:

- ☒ Loop
- ☒ Play On Awake

Volume:

0.4–0.7

Done.

Step 12 — Lighting

Directional Light:

Intensity:

0.7

Color:

slightly warm.

Shadows:

Soft Shadows

Ambient light:

slightly blue.

Peaceful feel.

Step 13 — Add Meditation Seat

Create cylinder / stone.

Place center.

Rename:

MeditationSpot

User viewpoint here.

Step 14 — Add XR Rig

Hierarchy →

Right click →

XR →

XR Origin

Place:

Meditation spot.

This becomes VR camera.

Step 15 — Camera Position

Set XR Origin:

X=0 Y=2 Z=0

User stands naturally.

Step 16 — Play

Press 

Scene:

 Sky

 Trees

 Waterfall

 Birds + water sounds

 Nature

 VR camera

Meditation environment complete.

Optional Improvements

Add:

- ✓ fog
- ✓ butterflies (particles)
- ✓ sun rays
- ✓ slow river animation
- ✓ temple bell sound

Looks amazing.

Viva Questions

Q1 What is Skybox?

Background environment around scene.

Q2 What is XR Origin?

VR player rig.

Q3 Why Audio Source?

Plays sound.

Q4 Why Terrain?

Natural landscape creation.

Q5 Why Soft Shadows?

Realistic peaceful lighting.

This practical is one of the best showcase projects because it combines:

- ✓ VR
- ✓ Environment design
- ✓ Sound
- ✓ Lighting
- ✓ Nature effects



Implement collision between any player character and the game world

Here is **Practical 10: Implement collision between player character and game world in Unity**, step by step for beginners.

Goal:

Character should:

- ✓ walk around
- ✓ collide with walls/objects
- ✓ not pass through table/rock/wall

- ✓ stand on ground
- ✓ detect collision

This demonstrates Unity Physics + Collision.

Step 1 — Create Project

Open Unity Hub

Create:

- **3D Core**

Project name:

PlayerCollision

Click **Create**

Step 2 — Create Ground

Top menu:

GameObject → **3D Object** → **Plane**

Rename:

Ground

Position:

0,0,0

Scale:

1,1,1

Step 3 — Create Player

GameObject →

3D Object → **Capsule**

Rename:

Player

Position:

- X = 0
- Y = 1

- $Z = 0$

Capsule is character.

Step 4 — Add Rigidbody

Select Player.

Inspector →

Add Component →

Search:

Rigidbody

Add.

Set:

Use Gravity = ON

Freeze Rotation:

check:

- X
- Z

(so player won't fall sideways)

Done.

Step 5 — Add Collider

Capsule already has:

Capsule Collider

Keep it.

Collision ready.

Step 6 — Create Walls

Create Cube.

Rename:

Wall1

Position:

0,1,10

Scale:

10,2,1

Duplicate walls:

Make boundary around player:

Front

Back

Left

Right

These act as world borders.

Cube already has Box Collider.

Done.

Step 7 — Create Obstacles

Create:

Cube → table

Sphere → rock

Cylinder → pillar

Place around scene.

All have colliders.

Player will collide.

Step 8 — Create Movement Script

Project →

Create C# Script:

PlayerMove

Paste:

```
C#
using UnityEngine;

public class PlayerMove : MonoBehaviour
{
    public float speed = 5f;

    Rigidbody rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");
```

```

    Vector3 move =
        new Vector3(h, 0, v) * speed;

    rb.velocity =
        new Vector3(move.x,
            rb.velocity.y,
            move.z);
}

void OnCollisionEnter(Collision collision)
{
    Debug.Log("Collided with: " + collision.gameObject.name);
}
}

```

Save.

Attach to Player.

Done.

Step 9 — Camera

Main Camera:

Position:

- X = 0
- Y = 10
- Z = -10

Rotation:

- X = 35

Can see full area.

Step 10 — Play

Press 

Move:

WASD / Arrow Keys

Result:

Character moves.

If touches:

Wall → stops

Rock → stops

Table → stops

Ground → stands

Cannot pass through objects.

Collision implemented.

Step 11 — Collision Detection

When collision happens:

Console prints:

`Collided with Wall1`

or

`Collided with Rock`

This confirms collision.

How collision works

Need:

Rigidbody

for physics

-

Collider

for collision boundary

Together = collision detection

Viva Questions

Q1 What is Collider?

Boundary for collision.

Q2 What is Rigidbody?

Adds physics.

Q3 What is OnCollisionEnter()?

Runs on collision.

Q4 Why Freeze Rotation?

Prevent tipping.

Q5 Difference: Trigger vs Collider?

Trigger passes through, collider blocks.

This is a very common Unity practical because collision is fundamental in games.